

חיפוש מיועד Heuristic Search

מה זה יוריסטיקה?

הגדרה: יוריסטיקה היא פונקציה שמעריכה את המרחק ממצב ליעד.

נדבר על יוריסטיקה כמספר טבעי $h : \Sigma \rightarrow \mathbb{N}_0 \cup \{\infty\}$

היוריסטיקה מקבלת את קודקוד החיפוש σ , אבל בד"כ היא צריכה רק את המצב s :

$$h(\sigma) := h(\text{state}(\sigma))$$

למה צריך יוריסטיקה?

- זה עובד טוב בפועל (גם אם לא משפר את הסיבוכיות האסימפטוטית)
- אם יש תכונות על היוריסטיקה, היא יכולה להבטיח דברים - למשל להחזיר פתרון אופטימלי.

יוריסטיקה מושלמת

הגדרה: יוריסטיקה אופטימלית או יוריסטיקה מושלמת h^* נותנת את המרחק האמיתי ליעד - או אינסוף עבור מבוי סתום.

משתמשים
בשביל
לסמן
דברים
אופטימליים

תכונות שיכולות להיות ליוריסטיקה

בטוחה: אם לכל $\sigma \in \Sigma$ המקיים $h(\sigma) = \infty$ אזי $h^*(\sigma) = \infty$. כלומר - היא לא יכולה להגיד על מצב שהוא מבוי סתום אם הוא לא מבוי סתום

מיועדת מטרות: אם $h(\sigma) = 0$ לכל $\sigma \in \Sigma$ המהווה קודקוד יעד. נשים לב שיכול להיות $h(\sigma) = 0$ גם עבור קודקודים שאינם יעד.

אדמיסיבילית: אם $h(\sigma) \leq h^*(\sigma)$ לכל $\sigma \in \Sigma$.

עקבית: אם $h(\sigma) \leq h(\sigma') + \text{cost}(\sigma, \sigma')$ לכל $\sigma, \sigma' \in \Sigma$ עבורם σ' בן של σ .

עיוורת: אם לכל $\sigma \in \Sigma$ מתקיים $h(\sigma) = 0$

נשים לב שיוריסטיקה עיוורת מקיימת את כל התכונות הקודמות - אבל היא לא עוזרת לנו. ועדיין - היא לא "תדפוק" לנו את האלגוריתם.

חיפוש חמדני best-first (GBFS)

האלגוריתם משתמש בערימת מינימום ממויינת לפי $h(\sigma) \mapsto \sigma$. לא מפתחים קודקודים עבורם $h(\sigma) = \infty$.

תכונות: • בהינתן יוריסטיקה בטוחה, האלגוריתם שלם: הוא יעבור על כל המצבים הנגישים מהמצב ההתחלתי.

• הוא לא אופטימלי - אלא אם כן היוריסטיקה מאוד טובה (מקיימת כמה תכונות מאוד חזקות)

A^* אלגוריתם

- ממין את הערימה לפי $f(\sigma) = g(\sigma) + h(\sigma)$ כאשר $\sigma \mapsto f(\sigma)$ הוא המחיר של קודקוד (החיפוש) σ .

- בנוסף לclosed שומרים מילון בשם distance שמעדכנים אותו:

$$\text{distance}(\text{state}(\sigma)) := g(\sigma)$$

– ברוב שפות התכנות אפשר לייצג אותם באותו מבנה נתונים (מי שאין לו distance הוא בהכרח לא closed כי עדיין לא הגענו אליו)

- מפתחים קודקודים בclosed אבל רק אם $g(\sigma) < \text{distance}(\text{state}(\sigma))$ (כלומר הגענו אליו בדרך יותר מהירה)

- תכונות: אם היוריסטיקה אדמיסבילית, הפתרון אופטימלי.

הוכחת אופטימליות

- טרמינולוגיה: ערך f של קודקוד - $f(\sigma) := g(\sigma) + h(\sigma)$

- קודקודים מיוצרים - שהוכנסו לopen בשלב מסוים

- קודקודים מפותחים (expanded) - שהוצאו מopen והבדיקה מול closed distance הצליחה

- קודקודים מפותחים מחדש - שפותחו בזמן שהיו בclosed

נניח בשלילה שקיבלנו פתרון לא אופטימלי עבור σ המקיים $\text{state}(\sigma) \in G$. אזי בהכרח אין קודקוד מטרה עם מסלול אופטימלי σ^* בopen.

הוכחה: h אדמיסבילית ולכן בהכרח $h(\sigma^*) \leq h^*(\sigma^*) = 0$ ולכן $f(\sigma^*) < f(\sigma)$ - כלומר היינו אמורים להוציא את $f(\sigma^*)$ קודם.

כמו כן, אחד מהקודקודים σ' במסלול האופטימלי מקודקוד ההתחלה σ_0 נמצא בהכרח בopen (שכן פיתחנו את σ_0). בגלל שהיוריסטיקה אדמיסבילית (ולכן $h(\sigma^*) = 0$):

$$f(\sigma') = g(\sigma') + h(\sigma') \leq g(\sigma^*) < g(\sigma) = f(\sigma)$$

כלומר בהכרח $f(\sigma) > f(\sigma')$ - ולכן האלגוריתם היה אמור להוציא את σ' לפני σ . קיבלנו סתירה.

A^* ממושקל

משתמשים ב

$$f(\sigma) = g(\sigma) + W \cdot h(\sigma)$$

כאשר $W \in \mathbb{R}_0^+$

נשים לב: • עבור $W = 0$ זה מתנהג כמו BFS

• עבור $W = 1$ זה מתנהג כמו A^*

• עבור $W \rightarrow \infty$ זה מתנהג כמו GBFS

עבור $W > 1$ האלגוריתם הזה הוא W -אופטימלי - כלומר העלות של הפתרון שהוא מוצא לא תהיה יותר מפי W מהפתרון האופטימלי:

$$opt^* \leq cost(\pi) \leq W \cdot opt^*$$

אלגוריתמים לא שלמים

Hill-climbing

לוקח בכל פעם את הבן עם היוריסטיקה הכי טובה (עם שובר-שוויון אקראי) - ומתעלם מכל האחרים.

נשים לב: • יכול להיות שהאלגוריתם לא יגיע למטרה וירוך לנצח - בפועל, עד שיגמר הזיכרון, שכן הוא עדיין מפתח עץ חיפוש וגם אם הוא חוזר על מצבים הוא צריך לשמור את כל המסלול.

• גם עבור יוריסטיקה כבילה יכול להיות שהוא לא ימצא פתרון. למשל - עבור יוריסטיקה עיוורת האלגוריתם יהיה random walk.

• עבור h^* האלגוריתם שלם - אבל לא בהכרח אופטימלי, שכן הוא מסתכל על היוריסטיקה של הקודקוד הבן אבל לא על העלות להגיע אליו.

- אבל עבור h^* ומחיר אחיד, הפתרון יהיה אופטימלי.

Enforced hill-climbing

בכל שלב, האלגוריתם מריץ BFS מהקודקוד σ_0 עד שהוא מוצא קודקוד σ עבורו $h(\sigma) < h(\sigma^0)$ ומחזיר אותו.

כלומר - במקום להסתכל על הבן עם היוריסטיקה הקטנה ביותר, מחפשים פשוט קודקוד עם יוריסטיקה טובה יותר.

תכונות: • אם הגרף חד כיווני, האלגוריתם לא יתקע - אבל יכול להיכשל במציאת הפתרון אם הוא מגיע ל σ_0 שהוא מבוי סתום.

• אם הגרף לא מכוון, האלגוריתם שלם אם $h(\sigma) = 0$ לכל קודקודי המטרה - ו $h(\sigma) > 0$ לכל קודקוד שאינו קודקוד מטרה.